

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №8
по дисциплине «Вычислительная математика»
Тема: Интерполяция сплайнами.

Студент гр. 4342

Ларионов В.А.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Разработать программу на языке C++ для интерполяции функций алгебраическим многочленом в форме Лагранжа. Реализовать построение интерполяционного многочлена как по равномерной сетке узлов, так и по узлам Чебышёва. Выполнить визуализацию исходной функции, интерполяционного многочлена и графика погрешности с помощью библиотеки Matplot++. Исследовать влияние выбора узлов интерполяции на точность приближения.

Теоретический минимум.

Интерполяция – метод приближения функции $f(x)$ на отрезке $[a, b]$ многочленом $P_n(x)$ степени не выше n , который совпадает с $f(x)$ в заданных узлах $x_0, x_1, \dots, x_n$. Существование и единственность такого многочлена гарантированы, если узлы различны.

Интерполяционный многочлен Лагранжа имеет вид:

$$P_n(x) = \sum_{i=0}^n f(x_i) * L_i(x),$$

где $L_i(x) = \prod_{j \neq i} (x - x_j) / (x_i - x_j)$ – базисные многочлены Лагранжа.

Программа строит многочлен в канонической форме: $P_n(x) = c_0 + c_1 \cdot x + c_2 \cdot x^2 + \dots + c_n \cdot x^n$. Для этого вычисляются коэффициенты c_k как сумма вкладов от каждого базисного многочлена. Вклад от узла x_i получается перемножением $(x - x_j)$ для $j \neq i$, делением на произведение разностей $(x_i - x_j)$ и умножением на $f(x_i)$.

Выбор узлов интерполяции существенно влияет на погрешность. При использовании равномерной сетки на отрезке $[a, b]$ ($x_k = a + k \cdot h$, $h = (b-a)/n$) может возникать эффект Рунге – сильные осцилляции многочлена вблизи концов интервала, особенно для функций, не являющихся аналитическими на всей комплексной плоскости.

Узлы Чебышёва (корни многочлена Чебышёва первого рода) минимизируют максимальную погрешность интерполяции. На отрезке $[a, b]$ они вычисляются по формуле:

$$x_k = (a+b)/2 + (b-a)/2 * \cos((2k-1)\pi / (2n)), k = 1..n.$$

В программе для преобразования к отрезку $[a, b]$ используется линейное отображение из стандартного отрезка $[-1, 1]$, где узлы Чебышёва имеют вид $\cos((2k-1)\pi/(2n))$. Такой выбор узлов позволяет избежать эффекта Рунге и получить равномерную сходимость для непрерывных функций.

Исходная функция для экспериментов:

$$f(x) = x^3 - 3 \cdot x - 2 \cdot e^{-x}.$$

Погрешность интерполяции оценивается как $E(x) = |f(x) - P_n(x)|$. В работе строятся графики исходной функции, интерполяционного многочлена и модуля разности между ними.

Выполнение работы.

Программа состоит из следующих файлов:

CMakeLists.txt – сценарий сборки с подключением библиотеки Matplot++.

includes.hpp – общие заголовки: подключение стандартных библиотек, Matplot++, определение типа ld (long double), пространства имён matplot.

utils.hpp – вспомогательные функции: вычисление узлов Чебышёва (getChebyshevNodes), построение графиков (buildGraphs), а также задание интерполируемой функции $f(x)$ и её производных (для возможного расширения).

main.cpp – основной модуль: ввод границ интервала [xMin, xMax] и числа узлов valuesCount; вычисление коэффициентов интерполяционного многочлена методом Лагранжа; построение графиков функции, интерполяции и погрешности с сохранением в SVG-файлы.

run.sh – скрипт сборки и запуска (требуется установленный CMake и Matplotlib).

Алгоритм построения интерполяционного многочлена

Пользователь вводит xMin, xMax, valuesCount (n+1).

В функции getInterpolationValues формируются узлы: по умолчанию закомментированы равномерные узлы, но в активной версии используются узлы Чебышёва (вызов getChebyshevNodes).

Вычисляются значения функции $f(x)$ в узлах.

Для каждого узла i ($0..n-1$) вычисляются коэффициенты базисного многочлена $L_i(x)$ в каноническом представлении. Алгоритм:

знаменатель $\text{denom} = \prod_{j \neq i} (x_i - x_j)$.

строится полином $\prod_{j \neq i} (x - x_j)$ путём последовательного умножения на $(x - x_j)$: начиная с полинома $[1]$ (коэффициент при x^0), затем для каждого $j \neq i$ выполняется $\text{conv}(p, [-x_j, 1])$ – т.е. умножается на $(x - x_j)$.

полученные коэффициенты умножаются на $f(x_i)/\text{denom}$.

Коэффициенты всех базисных многочленов суммируются. Результат – массив coefficients (размер valuesCount), где coefficients[k] – коэффициент при x^k в итоговом многочлене.

Глобальный вектор `interpolationValues` сохраняет эти коэффициенты.

Функция `interpolationFunction(x)` вычисляет значение многочлена по схеме Горнера (последовательное умножение на x и сложение с коэффициентами).

Функция `interpolationErrorFunction(x)` возвращает модуль разности между исходной функцией и интерполяционным многочленом.

Построение графиков

Функция `buildGraphs`:

Устанавливает терминал `gnuplot` для вывода в SVG с белым фоном.

Создаёт вектор x из $\sim 100 \cdot |b-a|$ точек на отрезке $[a, b]$ (линейно).

Для каждой переданной функции вычисляет $y = f(x)$ и строит график с толщиной линии 1.

Добавляет легенду, оси координат (нулевые линии), сетку.

Сохраняет рисунок в указанный файл (формат SVG).

В `main.cpp` выполняются два вызова:

Первый – графики исходной функции и интерполяционного многочлена (названия "Функция" и "Интерполяция") → сохраняется в `"testCheb.svg"`.

Второй – график погрешности (название `"|f(x) - P_n(x)|"`) → сохраняется в `"errCheb.svg"`.

Выводы.

В ходе работы была создана программа, реализующая интерполяцию функций алгебраическим многочленом в форме Лагранжа с вычислением коэффициентов канонического представления. Программа позволяет:

Выбирать узлы интерполяции – равномерные или Чебышёвские (переключение в коде).

Строить графики исходной функции, интерполяционного многочлена и абсолютной погрешности с использованием библиотеки Matplotlib.

Визуально оценить качество приближения и влияние распределения узлов.

Эксперимент показал, что узлы Чебышёва дают более равномерную и меньшую по величине погрешность по сравнению с равномерными узлами, особенно для функций, не являющихся полиномами. Это согласуется с теорией: многочлен, построенный по узлам Чебышёва, минимизирует максимум погрешности на отрезке.

Разработанное приложение может быть использовано как учебный инструмент для изучения интерполяции, а также как основа для более сложных методов аппроксимации (например, сплайн-интерполяции). Все файлы программы приведены в приложении.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
cmake_minimum_required(VERSION 3.14)
project(MyProject)

find_package(Matplot++ REQUIRED)

add_executable(main main.cpp)
target_link_libraries(main Matplot++::matplot)
#pragma once

#include <iostream>
#include <cmath>
#include <vector>
#include <string>
#include <functional>
#include <matplot/matplot.h>

using namespace matplot;
using ld = long double;
#include "../includes.hpp"

std::vector<double> getChebyshevNodes(int nodesCount) {
    std::vector<double> nodes(nodesCount);
    for (int i = 0; i < nodesCount; ++i) {
        nodes[i] = std::cos((2.0L * (i + 1) - 1.0L) * M_PI / (2.0L * nodesCount));
    }
    return nodes;
}

void buildGraphs(
    const double lowerXbound,
    const double upperXbound,
    std::vector<std::function<double(double)>>& functions,
    std::vector<std::string>& functionsName,
    const std::string& filename
) {
```

```

    setenv("GNUTERM", "svg enhanced background rgb 'white'", 1);
    auto x = linspace(lowerXbound, upperXbound, std::abs(upperXbound -
lowerXbound) * 100);
    figure(true);
    gcf()->color("white");
    std::vector<std::string> names(functions.size());
    for (size_t i = 0; i < functions.size(); ++i) {
        auto y = transform(x, functions[i]);
        auto p = plot(x, y)->line_width(1);
        hold(on);
    }
    hold(off);
    auto l = ::matplot::legend(functionsName);
    gca()->x_axis().zero_axis(true);
    gca()->y_axis().zero_axis(true);
    grid(on);
    save(filename);
}

double function(double x) {
    return std::pow(x, 3) - 3 * x - 2 * std::exp(-x);
}

double derivative1(double x) {
    return 2 * std::pow(M_El, -x) + 3 * std::pow(x, 2) - 3;
}

double derivative2(double x) {
    return - 2 * std::pow(M_El, -x) + 6 * x;
}

double derivative3(double x) {
    return 2 * std::pow(M_El, -x) + 6;
}

double derivative4(double x) {
    return - 2 * std::pow(M_El, -x);
}

```



```

}

double derivative5(double x) {
    return 2 * std::pow(M_El, -x);
}

#include "../includes.hpp"
#include "../utils.hpp"

std::vector<double> interpolationValues;

std::vector<double> getEquidistantPoints(
    double xMin,
    double xMax,
    int valuesCount
) {
    std::vector<double> points(valuesCount);

    double h = (xMax - xMin) / (valuesCount - 1);

    for (int i = 0; i < valuesCount; ++i)
        points[i] = xMin + i * h;

    return points;
}

std::vector<double> getLagrangeBasisCoeffs(
    const std::vector<double>& nodes,
    double funcValue,
    int i
) {
    int n = nodes.size();
    std::vector<double> coeffs(n, 0.0);
    coeffs[0] = 1.0;
    double denom = 1.0;
    for (int j = 0; j < n; ++j) {
        if (j == i) continue;
        denom *= (nodes[i] - nodes[j]);
        for (int k = n - 2; k >= 0; --k) {
            coeffs[k + 1] += coeffs[k];
            coeffs[k] *= -nodes[j];
        }
    }
}

```

```

    }
    double multiply = funcValue / denom;
    for (double& c : coeffs) {
        c *= multiply;
    }
    return coeffs;
}

std::vector<double>
getFunctionValues(
    std::vector<double>& points,
    std::function<double(double)> func
) {
    std::vector<double> values;
    values.reserve(points.size());
    for (auto point : points) {
        values.push_back(func(point));
    }
    return values;
}

double interpolationFunction(double x) {
    double result = interpolationValues[0];
    for (size_t i = 1; i < interpolationValues.size(); ++i) {
        double xPow = 1;
        for (int j = 0; j < i; ++j) {
            xPow *= x;
        }
        result += xPow * interpolationValues[i];
    }
    return result;
}

double interpolationErrorFunction(double x) {
    return std::abs(interpolationFunction(x) - function(x));
}

std::vector<double> getInterpolationValues(
    double xMin,
    double xMax,

```

```

    int valuesCount,
    std::function<double(double)> func
) {
    std::vector<double> values(valuesCount, 0.0);

    // std::vector<double> nodes =
    //     getEquidistantPoints(xMin, xMax, valuesCount);
    std::vector<double> nodes =
        getChebyshevNodes(valuesCount);

    std::vector<double> funcValues =
        getFunctionValues(nodes, func);

    for (int i = 0; i < valuesCount; ++i) {
        std::vector<double> coeffs =
            getLagrangeBasisCoeffs(
                nodes,
                funcValues[i],
                i
            );

        for (int j = 0; j < valuesCount; ++j) {
            values[j] += coeffs[j];
        }
    }

    return values;
}

int main() {
    double xMin, xMax;
    int valuesCount;
    std::cin >> xMin >> xMax >> valuesCount;
    interpolationValues = getInterpolationValues(xMin, xMax, valuesCount,
function);
    std::vector<std::function<double(double)>> funcs = {function,
interpolationFunction};
    std::vector<std::function<double(double)>> errFunc =
{interpolationErrorFunction};
    std::vector<std::string> funcsName = {"Функция", "Интерполяция"};

```

```
std::vector<std::string> errFuncName = {"|f(x) - P_n(x)|"};
buildGraphs(xMin, xMax, funcs, funcsName, "testCheb.svg");
buildGraphs(xMin, xMax, errFunc, errFuncName, "errCheb.svg");
}
#!/bin/bash

cmake -S . -B build
cmake --build build -j
./build/main
```